UrbanOmniDetect - Static Camera & BEV Optimization

Questa repository contiene una versione modificata e ottimizzata dello script `bev_realtime_v2.py` originale di UrbanOmniDetect. Le modifiche sono state pensate per scenari a **telecamera fissa** (es. monitoraggio di incroci o strisce pedonali), garantendo massima stabilità geometrica, assenza di artefatti 3D e alte prestazioni grazie all'inferenza TensorRT.

🛠️ Modifiche apportate a `bev_realtime_v2.py`

Per adattare la pipeline originale alle esigenze di monitoraggio metrico stabile, sono state implementate le seguenti modifiche core:

1. **Iniezione di Omografia Statica:** Il calcolo dinamico dell'omografia (che causa micro-vibrazioni del piano stradale) è stato bypassato. Lo script ora carica all'avvio una matrice pre-calibrata (`homography_bangkok.npy`), garantendo che le coordinate sulla Bird's-Eye View (BEV) siano metricamente costanti e immutabili.
2. **Correzione Anti-Farfalla (Riordino Angolare):** I keypoint 3D (sia a terra che sul tetto) estratti dalla rete neurale vengono riordinati angularmente rispetto al loro centroide in tempo reale. Questo previene l'effetto "farfalla" (poligoni auto-intersecanti) che si verificava quando i veicoli entravano parzialmente nell'inquadratura.
3. **Layout Grafico Blindato:** Risolto un bug di allocazione RAM fissando la risoluzione della dashboard a `1920x1400`. Il pannello inferiore per le scritte è ora garantito a prescindere dalle dimensioni del video di input.
4. **Barra di Progresso CLI:** Aggiunto il calcolo dei frame totali e una percentuale di avanzamento live nel terminale.

🚀 Guida all'Esecuzione

Per avviare la pipeline ottimizzata, utilizzare il seguente comando:

Shell

```
python bev_realtime_v2.py --input video_dataset_bangkok\video_041_tagliato_2.avi
--kp-model checkpoints\urbanomnidetect_v2_x_640.engine --kp-imgsz 640 --kp-conf
0.1 --kp-vis 0.5 --class-conf 0:0.4,1:0.5,3:0.5 --suppress-nested 0.85 --layout
dashboard --device cuda:0 --output output_videos\out_041_test_5.avi --object-px
22.0 --smooth 11 --fixed-camera on
```

📖 Spiegazione dei Parametri

- `--input`: Il percorso del video da processare.
- `--kp-model`: Il modello utilizzato. Abbiamo esportato il `.pt` nel formato ottimizzato `.engine` (TensorRT) per massimizzare gli FPS.
- `--kp-imgsz 640`: La risoluzione nativa di inferenza del modello.
- `--kp-conf 0.1` / `--kp-vis 0.5`: Soglie minime di confidenza per accettare una bounding box e per considerare validi i vertici 3D del cuboide.
- `--class-conf 0:0.4,1:0.5,3:0.5`: Soglie di confidenza specifiche per classe (ID COCO: 0=pedoni, 1=bici, 3=moto).
- `--suppress-nested 0.85`: Rimuove le detection duplicate (es. l'abitacolo rilevato come "auto" all'interno del box di un "camion").
- `--layout dashboard`: Attiva il layout 1920x1400 comprensivo di Video, colonna BEV e pannello testo inferiore.
- `--device cuda:0`: Forza l'esecuzione sulla GPU primaria.
- `--output`: Percorso in cui salvare il video renderizzato finale.
- `--smooth 11`: Attiva lo smoothing offline (Zero-Phase Filter) che analizza 11 frame alla volta (5 passati, 5 futuri) per rendere le traiettorie e i cuboidi fluidi e privi di micro-scatti.
- `--fixed-camera on`: **Fondamentale.** Spegne l'algoritmo di Optical Flow che cerca il movimento di sfondo. Risparmia enormi risorse di calcolo e blocca la scena confermando che la telecamera è fissa su palo.

🔎 Focus: `--object-px 22.0`

Questo parametro controlla lo **zoom fisico** all'interno della Bird's-Eye View (BEV).

- **Significato:** Indica che l'impronta di un veicolo mediano deve occupare esattamente `22.0` pixel sullo schermo della vista dall'alto.
- **Da dove deriva?** Il valore `22.0` non è casuale. È stato estratto preventivamente eseguendo lo script originale con il parametro di autocalibrazione `--bev-fit`. Quel tool ha misurato l'estensione massima di tutte le traiettorie dell'incrocio di Bangkok e ha calcolato che `22.0` è il livello di zoom perfetto per far rientrare l'intera carreggiata nel pannello laterale senza tagliare fuori nessun veicolo. Passandolo direttamente, evitiamo ricalcoli e otteniamo un'inquadratura perfetta fin dal primo frame.

Per eseguire il test che produce il video con le detection e la BEV a fianco:

python bev_realtime_v2.py --input video_dataset_bangkok\video_041_tagliato_2.avi --kp-model checkpoints\urbanomnidetect_v2_x_640.engine --kp-imgsz 640 --kp-conf 0.1 --kp-vis 0.5 --class-conf 0:0.4,1:0.5,3:0.5 --suppress-nested 0.85 --layout dashboard --device cuda:0 --output output_videos\out_041_test_5.avi --object-px 22.0 --smooth 11 --fixed-camera on

Per calibrare l'omografia da video:

python extract_homography.py --input video_dataset_bangkok\video_041.avi --kp-model checkpoints\urbanomnidetect_v2_x_640.engine --output homography_bangkok.npy --max-frames 3000

OBS: per fare questo utilizzare un video con elevato traffico, non tagliato ma in versione originale